The Mechanics of Red-Black Trees: A Visual Approach

Antonios Kordatzakis

July 29, 2026

1 Introduction

A *Red-Black* tree [1, 3] is a self-balancing binary search tree in which every *link* carries an additional *COLOR* attribute, taking the value *RED* or *BLACK*. This attribute serves to prevent the tree from becoming unbalanced. It should be noted that the *COLOR* attribute belongs to a *link* rather than to a *node* of the tree. As will be shown, when the *Red-Black* tree is modeled in the algorithms and the implementation, the absence of a class representing a *link* leads us to store the *COLOR* property in the tree node class. Accordingly, when a node is described as *RED* or *BLACK*, the intended meaning is that the link from its parent to that node is *RED* or *BLACK*.

The operations of a *Red-Black* tree are more readily understood when the tree is visualized as in [1], that is, as a tree in which:

- a *RED* link connects two nodes at the same level.
- a *BLACK* link increases the level by one.

Since the objective is to construct a balanced tree, two consecutive *RED* links are not permitted. A single *left RED* link and a single *right RED* link may originate from the same node, but the following link must be *BLACK*. For convenience, and to simplify the algorithms used to manipulate a *Red-Black* tree, the color of the root of the tree is set to *BLACK*, although an implementation may choose to disregard this convention.

In [4, 5] all *RED* nodes lean to the left. The present work adopts the original definition of *Red-Black* trees, which admits both left and right *RED* nodes.

Consider a *Red-Black* tree into which the following keys have been inserted: 11, 2, 14, 1, 7, 15, 5 and 8. The resulting tree, whose root node has key *11*, is shown in Figure 1.

All *leaf* nodes of the *Red-Black* tree above lie at the same level; the tree is therefore balanced.

In general, the construction of a self-balancing tree requires two mechanisms:

- a mechanism to propagate violations of balance to the level above, until no further violations remain — the *Propagation* mechanism.

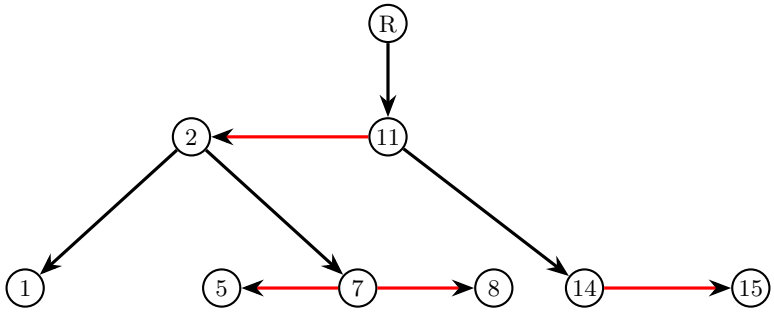


Figure 1: A Red-Black tree

- a mechanism to locally balance an unbalanced sub-tree — the *Rebalance* mechanism.

The *COLOR* property is used to create these two mechanisms in *Red-Black* trees.

2 Properties of the Red-Black trees

As noted above, for convenience the *COLOR* is modeled as an attribute of the node rather than of the link. The following rules apply to any *Red-Black* tree:

1. A node is either *RED* or *BLACK*; that is, the link towards this node is either *RED* or *BLACK*.
2. The *root* is *BLACK*.
3. A `NIL` leaf is *BLACK*.
4. Both children of a *RED* node should be *BLACK*.
5. Every simple path from *root* to a leaf has the same number of *BLACK* nodes.

3 Insertion

3.1 Fixing violations

When a new node is inserted into a *Red-Black* tree, it is always connected to its parent by a *RED* link. A *BLACK* link is not used, since it would alter the height of the tree, which is undesirable: correcting a height violation caused by the insertion of a *BLACK* node is a more difficult task than correcting violations arising from *RED* nodes. Furthermore, the height of the tree is kept to a minimum for as long as no violation is present. When there is no room for the new node and a new level must be introduced, this is achieved by moving

upwards and correcting any new violations that arise. In every case—whether during an insertion, a deletion, or a correction—the height of the tree is, where possible, kept unchanged.

No violation can occur when the parent of the new node is *BLACK*:

- no two consecutive *RED* nodes are created, since the parent is *BLACK*.
- the number of *BLACK* nodes remains unchanged, since the new node is *RED*.

It therefore suffices to examine the cases in which the parent of the new node is *RED*. To correct these violations, rotations such as those in [2] are not used; instead, the visual representation of the violation is allowed to guide the algorithms.

It should be borne in mind that, once a violation has been corrected, a new violation may arise at a higher level; consequently the corrections may have to be applied to an ancestor of the current node, until a *BLACK* link is reached.

This section examines the three distinct types of violation that arise when working with *Red-Black* trees. Together with their three symmetrical counterparts, this yields a total of six distinct types of violation.

The term "uncle" of a node denotes the sibling of the node's parent [1]. The color of the uncle determines how the violation is corrected.

3.1.1 Case 1: Red uncle — Propagation mechanism

Suppose we are given a *Red-Black* tree with nodes (11), (2) and (14), together with the parent of node (11), denoted (p) [2] ; see Figure 2.

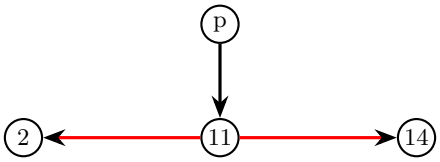


Figure 2: The initial tree

The link $p \to 11$ must be *BLACK*; otherwise there would be two consecutive *RED* links, which is invalid. If node (p) has height h, then node (11) has height $h+1$, which is also the height of nodes (2) and (14).

Consider now the insertion of node (1) shown in Figure 3. The two consecutive *RED* links $11 \to 2$ and $2 \to 1$ constitute a violation that must be corrected. The uncle of node (1) is node (14), which is *RED*. A new level is therefore introduced. In *Red-Black* trees, levels are always created by moving upwards; consequently node (11) is raised by one level. As a result of introducing this

[1] Equivalently, the child of the grand-parent that is not the parent of the node.

[2] The children of all nodes need not be shown, since the correction does not affect them.

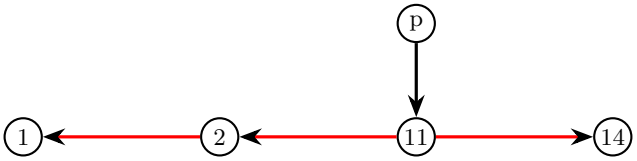


Figure 3: Inserting a new node — uncle is *RED*

level, the parent of node (11) has height h, node (11) has height $h+1$, and nodes (2) and (14) have height $h+2$. However, in order not to alter the level of the *leaf* nodes, the color of the link $p \rightarrow 11$ must be changed from *BLACK* to *RED*, which restores the heights of the nodes to their previous values (see Figure 4).

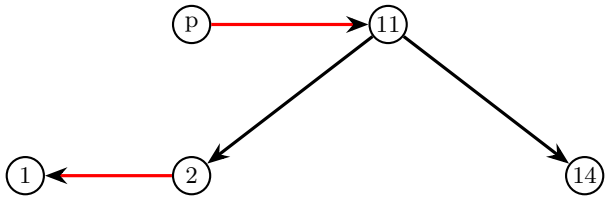


Figure 4: Fix of *Case 1* violation

Because the color of the link $p \rightarrow 11$ has been changed from *BLACK* to *RED*, it is necessary to verify whether a new violation has been introduced. Upon completion of this operation, the node under consideration becomes the grand-parent of the newly created node (11).

If node (11) is the *root* of the tree, then the link $p \rightarrow 11$ must be changed back to *BLACK*.

This correction realizes the *Propagation* mechanism of *Red-Black* trees: it propagates the balance violation to the level above, until a *BLACK* link or the *root* of the tree is reached.

3.1.2 Case 2: Black uncle — Rebalance mechanism

Consider now the case in which the uncle of an invalid node is *BLACK*. Two cases arise, according to whether the new node is the *left* or the *right* child of its parent.

3.1.3 Case 2.1: Right child

Suppose that keys have been inserted into a *Red-Black* tree and that the node under consideration is node (7), which is the right child of its parent, node (2) Figure 5. The uncle of node (7) is node (14), which is a *BLACK* node. Both cases implement the *Rebalance* mechanism of *Red-Black* trees.

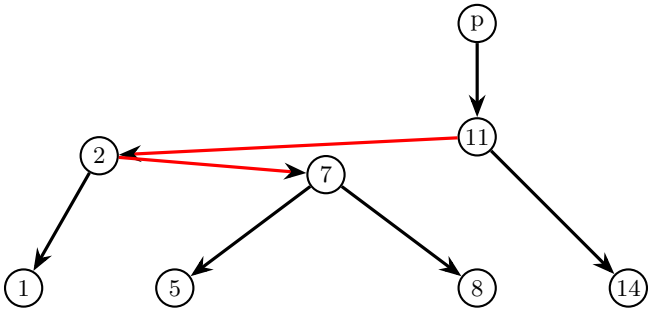


Figure 5: Inserting a new node — zig-zag, uncle is *BLACK*

This is clearly not a valid *Red-Black* tree, on account of the two consecutive *RED* links $11 \rightarrow 2$ and $2 \rightarrow 7$; it may, however, be regarded as a temporary arrangement arising while other violations are being corrected as one moves upwards, until a *BLACK* link is reached. It is readily seen that the tree "leans" to the left, a condition that requires correction.

Observe that node (7) naturally belongs between node (2) and node (11). Since node (7) will connect node (2) and node (11), its current children must be detached. At the same time node (2) loses its right child and node (11) loses its left child. In order to preserve the binary search tree property, node (2) is connected to node (5) and node (11) to node (8).

Accordingly, this violation is corrected as follows:

1. Create a *RED* link $7 \rightarrow 2$
2. Create a *RED* link $7 \rightarrow 11$
3. Create a *BLACK* link $2 \rightarrow 5$
4. Create a *BLACK* link $11 \rightarrow 8$
5. Create a *BLACK* link $p \rightarrow 7$

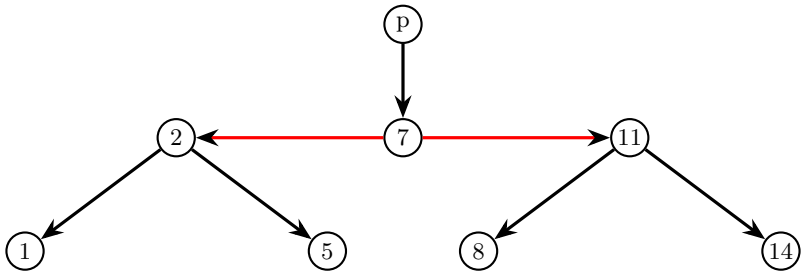


Figure 6: Fix of *Case 2.1* violation

Upon completion of this operation, node (7) becomes the new root, and no new violation exists, since the root of the sub-tree under consideration is connected by a *BLACK* link.

The resulting sub-tree is now balanced (Figure 6).

3.1.4 Case 2.2: Left child

Consider now the second case, in which the node under consideration is the left child of its parent. Suppose this node is node (2), the left child of its parent, node (7). The uncle of node (2) is node (14), which is a *BLACK* node.

As before, the tree in Figure 7. "leans" to the left, a condition that requires correction.

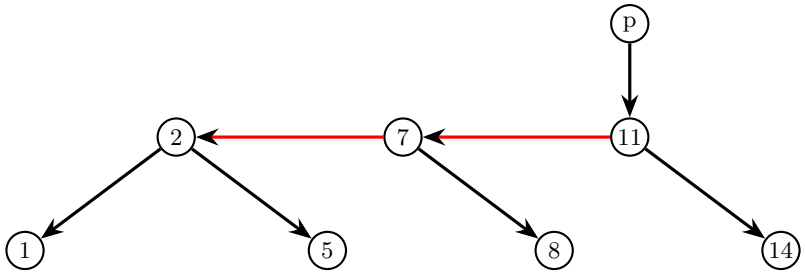


Figure 7: Inserting a new node — left child, uncle is *BLACK*

The direction of the link 11 → 7 is reversed to 7 → 11. Since node (7) becomes connected to node (11), its right child is detached, and at the same time node (11) loses its left child. In order to preserve the binary search tree property, node (11) is connected to node (8).

Accordingly, this violation is corrected as follows:

1. Delete the link 7 → 8
2. Change the direction of link 11 → 7 to 7 → 11
3. Create a *BLACK* link 11 → 8

Upon completion of this operation, all violations have been corrected, since a *BLACK* link has been reached; nothing further remains to be done.

Once again, the resulting sub-tree is now balanced (Figure 8).

4 Deletion

4.1 Case D.1. *BLACK* sibling with a right *RED* child

See the Figure 9a, or maybe Figure 9.

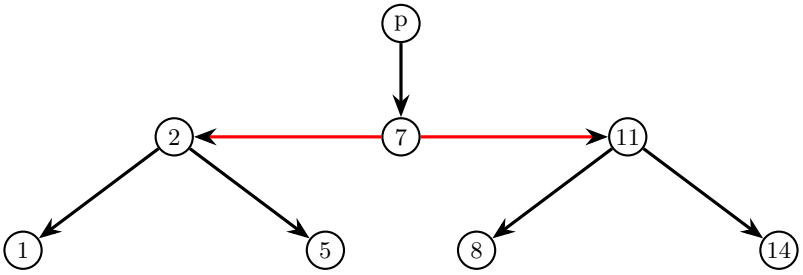
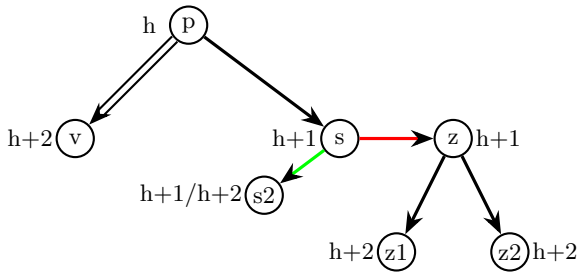
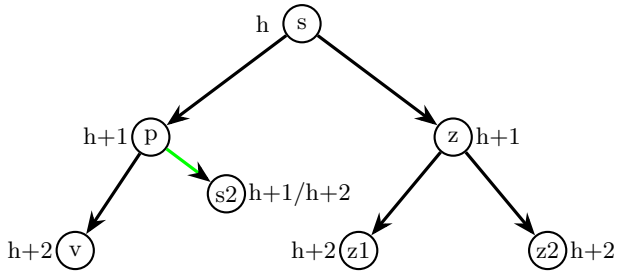


Figure 8: Fix of *Case 2.2* violation



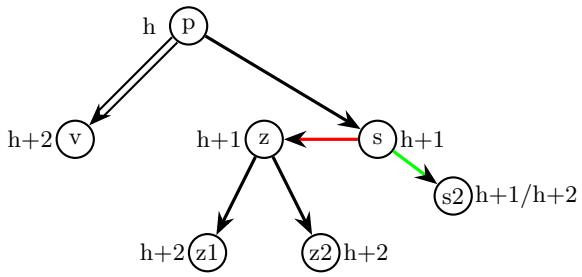
(a) *Case D.1*



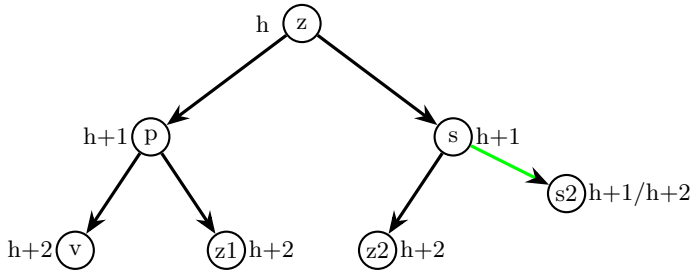
(b) Fix of *Case D.1*

Figure 9: *Case D.1*: *BLACK* sibling with a right *RED* child

4.2 Case D.2. *BLACK* sibling with a left *RED* child



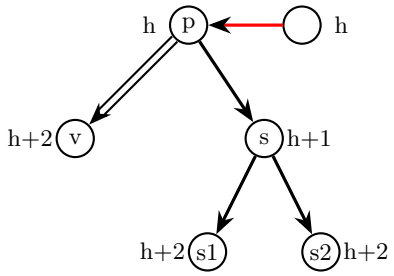
(a) *Case D.2*



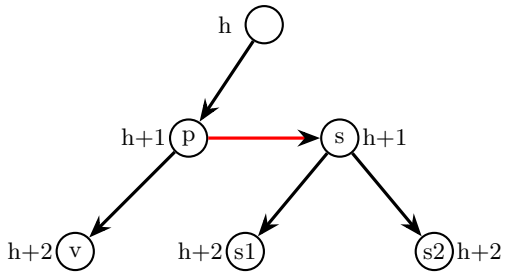
(b) Fix of *Case D.2*

Figure 10: *Case D.2*: *BLACK* sibling with a left *RED* child

4.3 Case D.3. *BLACK* sibling with *BLACK* children and *RED* parent



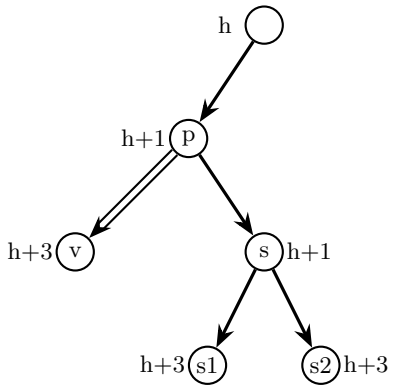
(a) *Case D.3*



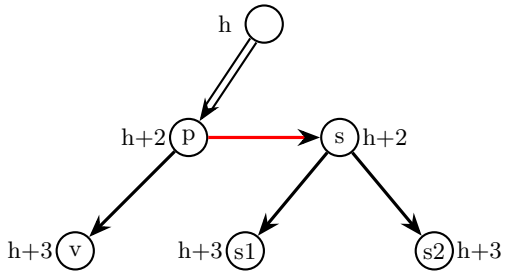
(b) Fix of *Case D.3*

Figure 11: *Case D.3*: *BLACK* sibling with *BLACK* children and *RED* parent

4.4 Case D.4. *BLACK* sibling with *BLACK* children and parent



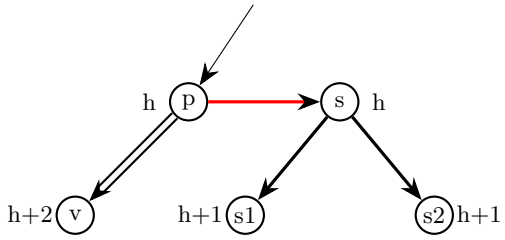
(a) *Case D.4*



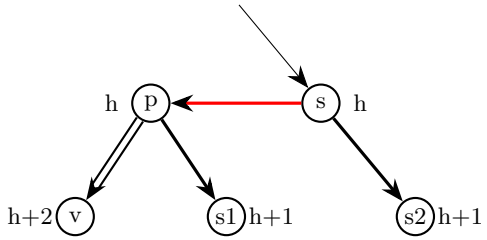
(b) Fix of *Case D.4*

Figure 12: *Case D.4*: *BLACK* sibling with *BLACK* children and parent

4.5 Case D.5. *RED* sibling



(a) *Case D.5*



(b) Handling of *Case D.5*

Figure 13: *Case D.5*: *BLACK* sibling with *BLACK* children and *BLACK* parent

5 Implementation

In the source code listings of the remaining sections, a hypothetical language (pseudo-code) resembling `C#` or `Java` is used.

5.1 Type RedBlackTree

The type `RedBlackTree` contains at least these three functions:

```
public class RedBlackTree<K, V>
    where K: Comparable
{
    void Insert(K key, V value);
    bool Delete(K key);
    V? Search(K key);
}
```

The `Delete` function returns *true* if the key was found in the tree and the corresponding node was deleted, and *false* otherwise. The `Search` function returns `NIL` when the *key* is not found in the tree.

Optionally, `RedBlackTree` may provide a read-only `Size` property of type `Integer`, which returns the number of keys in the tree.

5.1.1 Comparing keys

The `Comparable` interface contains a function that compares this key with another:

```
public interface Comparable<K>
{
    int CompareTo(K other);
}
```

If the implementation language does not support interfaces, a simple function comparing the two keys is sufficient, for example:

```
int compare_to(K key1, K key2)
```

In either case, the function should return:

- 0, if the two keys are equal.
- a value <0, if *key1* is less than *key2*.
- a value >0, if *key1* is greater than *key2*.

5.2 Type Node

Every node instance of a *Red-Black* tree is of type `Node` that contains the following properties:

- *Key*, of a template type `K`.
- *Value*, of a template type `V`.
- *Color*, of a type `COLOR` an enumeration with values `RED` or `BLACK`.
- *Left*, *Right* of type `Node` reference.

5.2.1 The parent node

In many implementations the `Node` type contains a reference to its *Parent* node. Since this increases the memory consumption of the data structure and complicates the algorithms presented here, a different approach is adopted: the ancestors of a node are determined by storing the path from the *root* to the position at which the new node is to be inserted, as the tree is traversed. The *parent*, *grand-parent* and *great-grand-parent* of a node are simply references within the `Path` type, a specialized *Stack* data structure initialized and populated in the `Insert` function.

5.2.2 Child direction

Some algorithms make a decision based on whether a node is the left or the right child of its parent. There is no need to store any additional information in each node (such as a property `bool IsLeftChild`), since whether a node is the left or the right child of its parent can readily be determined by comparing it with the `Left` or the `Right` property of the *Parent*, provided that the *Parent* is not `NIL`, that is, that the node is not the *root*.

5.2.3 Inserting the same key

The implementation should handle the case of inserting a key-value pair whose key already exists in the tree. The implementation may choose to:

- skip the insertion.
- replace the existing value of the key with the new value.
- append the new value to a collection of values associated with the node for this key, typically only if the value is not already present in the collection. In this case each node stores a collection of values rather than a single value, and the `Search` function must return a collection of values when the key is found.

In the reference implementation, the *value* of the node with the matching *key* is replaced.

5.2.4 The Sentinel node

To simplify the handling of boundary conditions, rather than using *NIL* references for leaf nodes or the root node, a *sentinel* node is used to represent a *NIL* node:

```
public class RedBlackTree
{
    // ...
    const Node Nil = new Node { Color = Black };
}
```

The implementation may assign any suitable values to the `Key`, `Value`, `Left`, `Right` and `Parent` properties. Only the `Color` property is explicitly set to *BLACK*. In addition, the `Key` property of the *Nil* node is set to the key used when searching.

It should be noted that not only all leaves but also the *root* node are set to point to the `Nil` node. This is consistent with the second rule of *Red-Black* trees, which requires the *root* node to be *BLACK*.

6 Searching

The recursive implementation of the `Search` function is:

```
public V? Search(K key)
{
    // Set the key of the Nil node to the value of
    // the key that we search. The value of the Nil
    // node is the default V value
    Nil.Key = key;

    return Search(Root, key);
}

private static V? Search(Node node, K key)
{
    var compare = key.CompareTo(node.Key);

    if (compare == 0) return node.Value;
    if (compare < 0) return Search(node.Left, key);
    return Search(node.Right, key);
}
```

Note that, because the sentinel is used in the private `Search` function, there is no need to test whether a node is *NIL*, thereby avoiding an additional condition in each recursive call. Note further that the node found may be the *Nil* node, in which case the value returned is the value of the *Nil* node, that is, *NIL*.

The iterative version of the `Search` function is:

```
public V? IterativeSearch(K key)
{
    // Set the key of the Nil node to the value of
    // the key that we search. The value of the Nil
    // node is the default V value
    Nil.Key = key;

    int compare;
    var node = Root;
    while ((compare = key.CompareTo(node.Key)) != 0)
    {
        if (compare < 0) node = node.Left;
        else node = node.Right;
    }

    return node.Value;
}
```

7 Insert

Using the techniques described in the previous sections, the insertion of a new key-value pair is straightforward. First, the position at which the new node is to be placed is located. During this traversal, the *path* is initialized and populated, which subsequently enables the ancestors of the new node to be determined.

Any two consecutive *RED* nodes violations that may occur are then corrected, moving upwards for as long as the current node is *RED*. At each iteration, the *Propagation* and *Rebalance* mechanisms described in the previous sections are applied.

```
public void Insert(K key, V value)
{
    var x = Root;
    var y = Nil;
    int compare = 0;

    var path = new Path();
    path.Push(Nil);

    while (x != Nil)
    {
        path.Push(x);

        y = x;
        compare = key.CompareTo(x.Key);

        if (compare == 0)
        {
            x.Value = value;
            return;
        }

        if (compare < 0) x = x.Left;
        else x = x.Right;
    }

    var z = new Node<K, V>(key, value, Nil);

    if (y == Nil) Root = z;
    else if (compare < 0) y.Left = z;
    else y.Right = z;

    path.Push(z);
    InsertFixup(path);
```

```
    Size++;
}

private void InsertFixup(Path path)
{
    Node<K, V> p;

    while ((p = path.PeekParent()).IsRed)
    {
        var z = path.Peek();
        var gp = path.PeekGrandParent();

        if (p.IsLeftChildOf(gp))
        {
            var uncle = gp.Right;

            if (uncle.IsRed)
            {
                // Case 1
                p.Color = Color.Black;
                uncle.Color = Color.Black;
                gp.Color = Color.Red;
                path.Pop();
                path.Pop();
            }
            else
            {
                if (z.IsRightChildOf(p))
                {
                    // Case 2.1
                    if (Root == gp)
                    {
                        Root = z;
                    }
                    else
                    {
                        var gpp =
                            path.PeekGreatGrandParent();
                        if (gp.IsLeftChildOf(gpp))
                            gpp.Left = z;
                        else
                            gpp.Right = z;
                    }

                    p.Right = z.Left;
                    gp.Left = z.Right;
```

```
                z.Left = p;
                p.Color = Color.Red;
                z.Right = gp;
                gp.Color = Color.Red;
                z.Color = Color.Black;
                break;
            }
            else
            {
                // Case 2.2
                if (Root == gp)
                {
                    Root = p;
                }
                else
                {
                    var gpp =
                        path.PeekGreatGrandParent();
                    if (gp.IsLeftChildOf(gpp))
                        gpp.Left = p;
                    else
                        gpp.Right = p;
                }

                gp.Left = p.Right;
                p.Right = gp;
                gp.Color = Color.Red;
                p.Color = Color.Black;
                break;
            }
        }
    }
    else
    {
        var uncle = gp.Left;

        if (uncle.IsRed)
        {
            // Case 1
            p.Color = Color.Black;
            uncle.Color = Color.Black;
            gp.Color = Color.Red;
            path.Pop();
            path.Pop();
        }
        else
```

```
{
    if (z.IsLeftChildOf(p))
    {
        // Case 2.1
        if (Root == gp)
        {
            Root = z;
        }
        else
        {
            var gpp =
                path.PeekGreatGrandParent();
            if (gp.IsLeftChildOf(gpp))
                gpp.Left = z;
            else
                gpp.Right = z;
        }

        p.Left = z.Right;
        gp.Right = z.Left;
        z.Right = p;
        p.Color = Color.Red;
        z.Left = gp;
        gp.Color = Color.Red;
        z.Color = Color.Black;
        break;
    }
    else
    {
        // Case 2.2
        if (Root == gp)
        {
            Root = p;
        }
        else
        {
            var gpp =
                path.PeekGreatGrandParent();
            if (gp.IsLeftChildOf(gpp))
                gpp.Left = p;
            else
                gpp.Right = p;
        }

        gp.Right = p.Left;
        p.Left = gp;
```

```
                    gp.Color = Color.Red;
                    p.Color = Color.Black;
                    break;
                }
            }
        }
    }

    Root!.Color = Color.Black;
}
```

8 Conclusion

This paper has sought to explain the mechanisms underlying *Red-Black* trees in a visual manner. It is argued that the true meaning of the rotations used in *Red-Black* trees is harder to grasp, and that the *Propagation* and *Rebalance* mechanisms are more readily assimilated in a visual manner by a reader encountering *Red-Black* trees for the first time.

References

[1] Rudolf Bayer. "Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms". In: *Acta Informatica* 1.4 (1972), pp. 290–306. DOI: 10.1007/BF00289509.

[2] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. Cambridge, MA: MIT Press, 2022. ISBN: 9780262046305.

[3] Leo J. Guibas and Robert Sedgewick. "A Dichromatic Framework for Balanced Trees". In: *19th Annual Symposium on Foundations of Computer Science (SFCS 1978)*. IEEE, 1978, pp. 8–21. DOI: 10.1109/SFCS.1978.3.

[4] Robert Sedgewick. *Left-Leaning Red-Black Trees*. https://sedgewick.io/wp-content/themes/sedgewick/papers/2008LLRB.pdf. Accessed: 2026-07-22. 2008.

[5] Robert Sedgewick and Kevin Wayne. *Algorithms*. 4th ed. Boston, MA: Addison-Wesley, 2011. ISBN: 9780321573513.